

B.Tech/M.Tech (Integrated) DEGREE EXAMINATION, JULY 2025

Sixth Semester

21CSC304J - COMPILER DESIGN*(For the candidates admitted during the academic year 2021-2022 to 2024-2025)***Note:**

- i. **Part - A** should be answered in OMR sheet within first 40 minutes and OMR sheet should be handed over to hall invigilator at the end of 40th minute.
- ii. **Part - B** and **Part - C** should be answered in answer booklet.

Time: 3 hours**Max. Marks: 75****PART - A (20 × 1 = 20 Marks)**

Answer all Questions

Marks BL CO

- | | | | |
|--|---|---|---|
| 1. Which of the following is NOT a phase of a compiler?
(A) Lexical Analysis (B) Parsing
(C) Debugging (D) Code Generation | 1 | 1 | 1 |
| 2. What is the role of a lexical analyzer in a compiler?
(A) To check syntax errors (B) To convert high-level code into machine code
(C) To recognize tokens from source code (D) To optimize intermediate code | 1 | 1 | 1 |
| 3. Which of the following tools helps in the construction of a lexical analyzer?
(A) YACC (B) LEX
(C) Linker (D) Assembler | 1 | 1 | 1 |
| 4. Why is input buffering used in lexical analysis?
(A) To reduce the overhead of reading characters one by one (B) To parse tokens efficiently
(C) To generate optimized code (D) To handle syntax errors | 1 | 2 | 2 |
| 5. Why is left recursion eliminated in top-down parsing?
(A) To simplify the grammar structure (B) To prevent infinite recursion during parsing
(C) To speed up code generation (D) To make parsing tables smaller | 1 | 2 | 2 |
| 6. What is the significance of the FIRST and FOLLOW sets in predictive parsing?
(A) They help in constructing the parse tree (B) They assist in error recovery
(C) They determine the correct production rules in LL(1) parsing (D) They optimize the lexical analysis phase | 1 | 2 | 2 |
| 7. How does a predictive parser handle ambiguity in a grammar?
(A) By rewriting the grammar to remove ambiguity (B) By backtracking through multiple parsing choices
(C) By ignoring ambiguous productions (D) By modifying tokens dynamically | 1 | 2 | 2 |
| 8. What is the role of an error recovery mechanism in predictive parsing?
(A) To automatically correct syntax errors (B) To stop parsing when an error is encountered
(C) To provide a way to continue parsing despite errors (D) To optimize the code generation process | 1 | 2 | 2 |

9. In a shift-reduce parser, what happens when a handle is detected on the stack? 1 2 3
 (A) The parser shifts the next input symbol (B) The handle is reduced using a grammar rule
 (C) The parser backtracks to the previous state (D) The parser stops execution
10. Which of the following grammar modifications would help in applying an LR parser? 1 3 3
 (A) Introducing left recursion (B) Removing ambiguity and left recursion
 (C) Converting it into regular expressions (D) Replacing terminals with non-terminals
11. You are constructing an SLR parsing table for a given grammar. If a conflict arises in the ACTION table, what should you do? 1 3 3
 (A) Modify the grammar to remove ambiguity (B) Add more states to handle conflicts
 (C) Convert the SLR parser into an LL(1) parser (D) Increase the lookahead to resolve conflicts
12. If a shift-reduce parser encounters a shift/reduce conflict, what should be done? 1 2 3
 (A) Modify the grammar to remove ambiguity (B) Always prefer shifting over reducing
 (C) Always prefer reducing over shifting (D) Ignore the conflict and proceed
13. Consider the following expression: 1 3 4
 $(a + b) * (c - d)$
 Which of the following is the correct postfix notation?
 (A) $a b + c d - *$ (B) $a + b * c - d$
 (C) $a b c d - + *$ (D) $a b + * c d -$
14. Given the three-address code instruction: 1 3 4
 $t1 = a + b$
 $t2 = t1 * c$
 $t3 = t2 - d$
 Which of the following is the correct quadruple representation?
 (A) $(+, a, b, t1), (*, t1, c, t2), (-, t2, d, t3)$ (B) $(a, b, +, t1), (t1, c, *, t2), (t2, d, -, t3)$
 (C) $(+, t1, t2, a), (*, a, c, b), (-, d, t3, t2)$ (D) $(t1, a, b, +), (t2, c, t1, *), (t3, d, t2, -)$
15. Why is postfix notation preferred for intermediate code representation? 1 2 4
 (A) It removes the need for parentheses and operator precedence rules (B) It executes faster than other notations
 (C) It reduces the number of registers required (D) It directly generates machine code
16. What is the main advantage of using three-address code in compilers? 1 1 4
 (A) It directly translates to machine code without further processing (B) It simplifies optimization by breaking complex expressions into smaller steps
 (C) It eliminates the need for memory allocation (D) It ensures that code executes faster than other representations
17. Why is peephole optimization used in compilers? 1 2 5
 (A) To improve the readability of the code (B) To find and replace inefficiencies in small code sequences
 (C) To remove comments from the source code (D) To check for syntax errors in the code

- | | |
|---|--|
| 18. Which of the following is an important step in loop optimization? | 1 2 5 |
| (A) Removing all loops from the program | (B) Replacing loops with conditional statements |
| (C) Reducing the number of iterations or unnecessary calculations inside loops | (D) Converting loops into recursive functions |
| | |
| 19. What is the role of a flow graph in code optimization? | 1 1 5 |
| (A) It helps in debugging programs | (B) It represents the control flow of a program to identify optimization opportunities |
| (C) It stores variable values during execution | (D) It translates source code into machine code |
| | |
| 20. Which of the following is an example of function-preserving transformation? | 1 2 5 |
| (A) Replacing an expression with an equivalent one that executes faster | (B) Removing all comments from the code |
| (C) Changing variable names for better readability | (D) Splitting a large program into multiple small functions |

PART - B (5 × 8 = 40 Marks)

Answer all Questions

Marks BL CO

- | | |
|--|-----------------|
| 21. (a) Explain the different phases of a compiler with a neat diagram.
(OR)
(b) Describe the role of a lexical analyzer and explain input buffering techniques used in lexical analysis. | 8 1 1 |
| | |
| 22. (a) Consider the following grammar:
S → aAB
A → bA ε
B → c
Construct the FIRST and FOLLOW sets for all non-terminals and create a predictive parsing table.
(OR)
(b) Construct a Predictive Parsing Table for the following grammar:
E → TE'
E' → +TE' ε
T → FT'
T' → *FT' ε
F → (E) id | 8 3 2 |
| | |
| 23. (a) Explain Shift-Reduce Parsing with an example. What are the common conflicts in Shift-Reduce Parsing?
(OR)
(b) Construct the SLR(1) parsing table for the following grammar:
S → CC
C → cC d | 8 3 3 |
| | |
| 24. (a) Construct the syntax tree and generate the corresponding Three-Address Code for the expression:
P = (A + B) * (C - D)
(OR)
(b) Discuss the role of Register and Address Descriptors in Code Generation with an example. | 8 2 4 |
| | |
| 25. (a) What is Peephole Optimization? Explain its different techniques with examples.
(OR)
(b) Explain the importance of Data Flow Analysis in Compiler Optimization with an example. | 8 2 5 |

PART - C ($1 \times 15 = 15$ Marks)

Marks BL CO

Answer **any 1** Questions

- | | | | |
|---|----|---|---|
| 26. How followpos() and firstpos() helps in constructing DFA? Write the algorithm for the same. Construct a DFA for the Regular Expression $a.(a+b).a.b$ using direct method. | 15 | 2 | 1 |
| 27. Explain the Role of Runtime Environments in Compiler Design. Discuss Storage Organization and Activation Records. | 15 | 3 | 5 |

* * * * *

THE HELPERS